Data Architecture: Examples, Tools, and Design Decisions

Plan first, map dependencies, test assumptions, then implement!

DAEN 459 – Fall 2026

Alexander Abuabara



Outcomes

1. Compare three common data architectures and the constraints that favor each.
2. Explain what each tool does and identify credible alternatives.
3. Use architecture maps, dependency graphs, and lineage diagrams to uncover risks.
4. Record decisions, assumptions, and tests *before* building the entire pipeline.

Key message

A diagram is not just documentation:
it is a planning and dependency-checking instrument.

Plan the system before choosing products

Capture requirements

- Consumers and expected outputs
- Sources, ownership, formats, frequency
- Freshness requirements: daily, hourly, seconds
- Volume, growth, security, budget
- Recovery and data-quality expectations

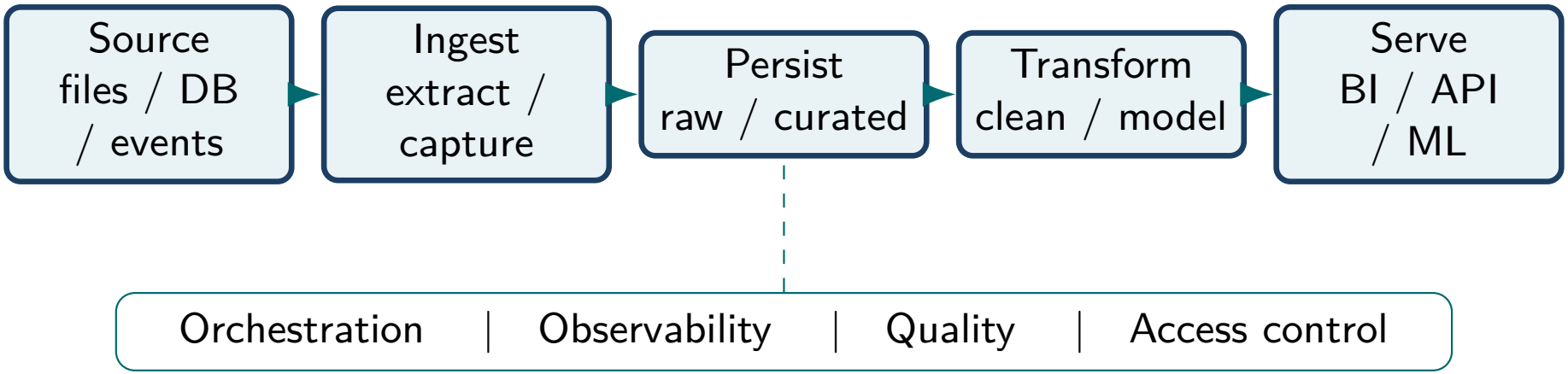
Create planning artifacts

1. Context diagram: systems and owners
2. Data flow: movement and storage
3. Dependency DAG: ordering and failure effects
4. Decision log: choices and alternatives
5. Thin vertical-slice test: one output end to end

Planning question

What has to be true for this design to work? Show where those assumptions are verified.

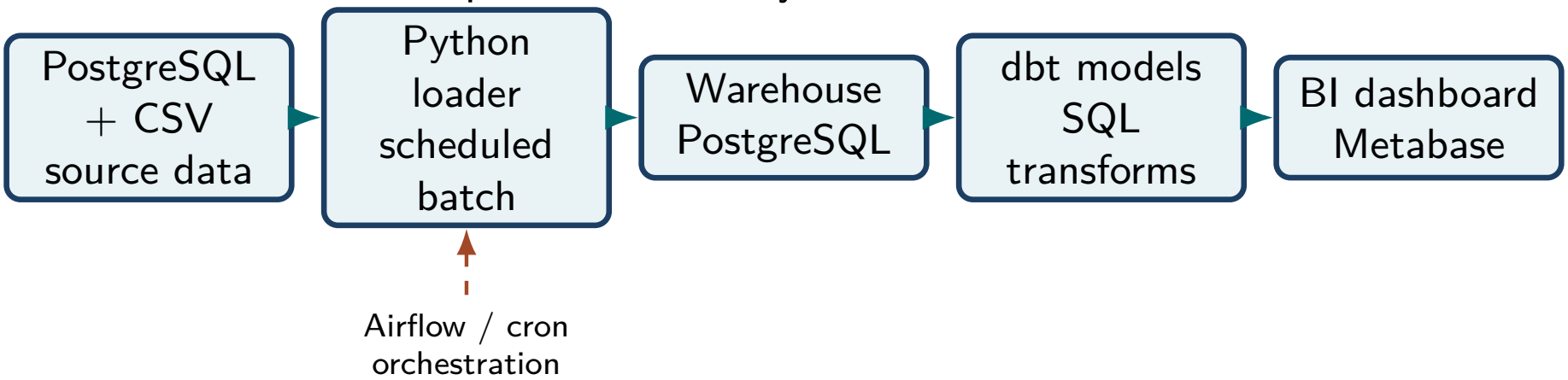
A reusable architectural vocabulary



- Separate **logical design** (required capabilities) from **physical design** (specific tools).
- Draw **data movement** and **control dependencies** separately when helpful.
- Every arrow implies a contract: schema, format, schedule, authentication, or reliability.

Example 1: Scheduled operational analytics (batch ELT)

Scenario: A campus unit combines transactional database data and CSV exports into a daily dashboard.



Why this architecture?

- Dashboard can tolerate daily freshness.
- SQL-centric transformations are auditable.
- Moderate scale supports a simple stack.

Critical dependencies

- CSV delivery before ingestion
- Loader success before dbt run
- Tests pass before dashboard refresh

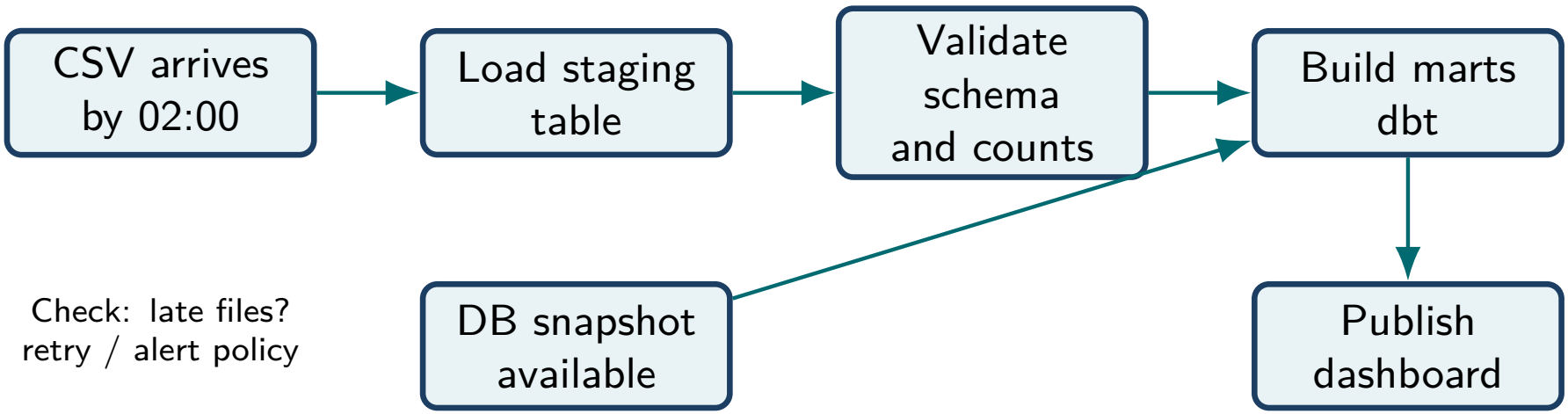
Example 1: Tool choices and alternatives

Function	Example tool	Alternatives and decision trigger
Ingestion	Python + SQL drivers	Airbyte for many connectors; managed ingestion if operations are constrained
Storage	PostgreSQL	DuckDB for single-machine analysis; BigQuery/Snowflake for cloud-scale multi-user workloads
Transforms	dbt (SQL)	Python/Pandas for custom code; Spark for distributed processing
Scheduling	cron or Airflow	Prefect/Dagster for richer workflows; cron if dependencies are trivial
Serving	Metabase	Power BI, Tableau, or custom application

Decision

Choose the **smallest manageable stack** that meets correctness, freshness, and maintainability needs. Airflow is not necessary for one independent daily script.

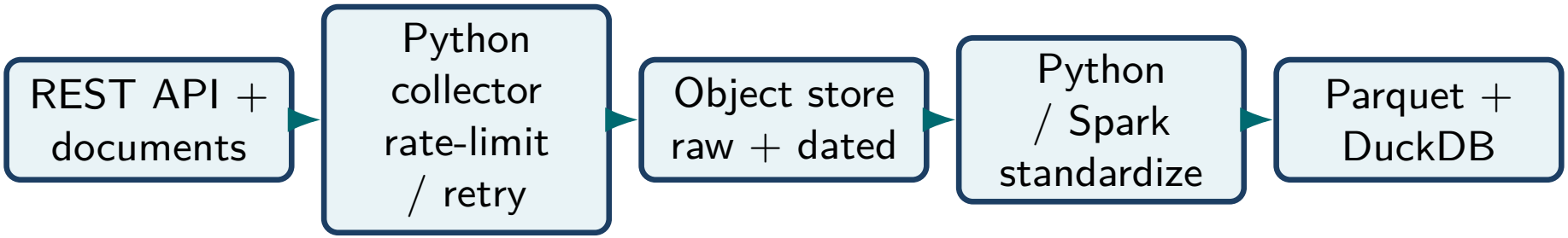
Example 1: Dependency map exposes failures



- The DAG shows **execution dependencies**, not just data movement.
- Identify gates: expected source arrival, schema tests, row-count checks.
- Define actions for late files, changed columns, and partial loads.

Example 2: API + object storage + analytical processing

Scenario: A research team ingests external API data and documents, preserves raw versions, and prepares reproducible datasets.



Architecture rationale

- Raw snapshots enable replay and auditing.
- Columnar files support repeatable analysis.
- Collection and processing can scale separately.

Planning checks

- API quotas and schema drift
- File naming/partition conventions
- Credentials, access and retention
- Reprocessing when logic changes

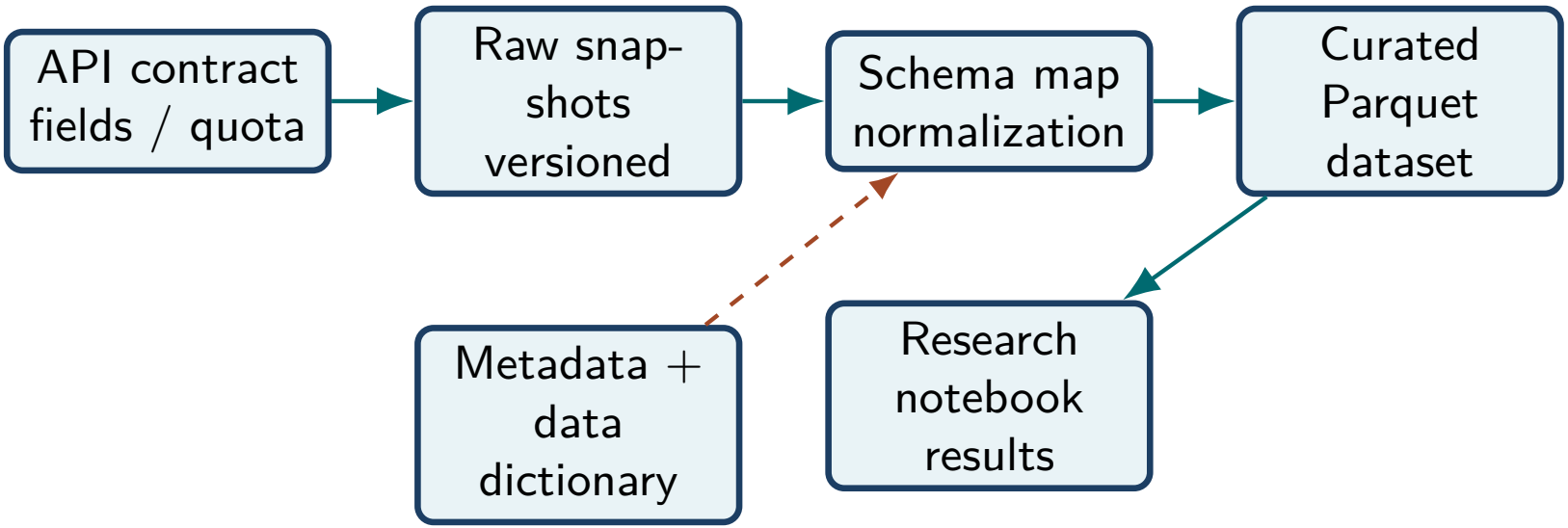
Example 2: Alternatives and trade-offs

Decision	Option A	Option B / selection evidence
Raw storage	Local files / NAS	S3 / Azure Blob / GCS: shared scalable storage, with cloud costs and access setup
Compute	Python / Polars / DuckDB	Spark: distributed work when one machine no longer meets measured targets
Curated format	Parquet files	Database tables: concurrency / updates / serving requirements may favor tables
Orchestration	Scheduled Python jobs	Prefect / Airflow / Dagster if retries, backfills, and dependencies grow

Decision evidence

Benchmark a representative dataset; document API limits, storage footprint, query time, reprocessing time, and access controls.

Example 2: Map upstream and downstream dependencies



- **Impact question:** If an API field changes, which curated columns and analyses break?
- Maintain a source-to-target mapping and document schema assumptions.
- Check whether full rebuilds are possible using saved raw data.

Example 3: Event-driven near-real-time monitoring

Scenario: IoT devices send measurements; the system detects abnormal conditions and exposes recent aggregates within seconds/minutes.



Rationale

- Events arrive continuously.
- Processing must handle delays and duplicates.
- Broker decouples producers and consumers.

Dependencies to map

- Event schema / device identity
- Broker health and retention
- Processor state / checkpointing
- Downstream alert availability

Example 3: Is streaming actually necessary?

Requirement	Simpler path	Streaming path
Freshness	Poll API / micro-batch every few minutes	Continuous Kafka + Flink when low latency is a measured requirement
Data movement	Direct write to DB / queue	Durable event broker for decoupling, replay, multiple consumers
Compute	Python worker / scheduled SQL	Stream processing for stateful windows, out-of-order events
Operations	Few services, simpler debugging	More components, deployments, monitoring, and recovery modes

Decision

Prototype polling first if the project can tolerate minutes of delay. Only adopt Kafka/Flink when measured latency, replay, or scale needs justify operating them.

Compare the three designs

Dimension	Batch warehouse	API + lake files	Event-driven streaming
Typical cadence	Daily / hourly	Periodic / on demand	Continuous
Main asset	Curated relational marts	Replayable raw snapshots + curated datasets	Durable event flow + current state
Biggest concern	Scheduling and stale data	API contracts and reproducibility	Ordering, duplicates, latency, operations
Good initial test	One full daily refresh	Ingest, store, and replay one partition	Simulate bursts and late events
Avoid by default	Excessive orchestration	Distributed compute too early	Complex brokers without an SLA

No architecture is "best" in isolation: the requirements and dependency risks determine the choice.

Four complementary maps teams should maintain

1. **System context:** upstream providers, users, organizational boundaries, credentials.
2. **Data flow / lineage:** datasets and fields, transforms, materializations, consumers.
3. **Execution dependency DAG:** scheduling, prerequisites, retries, backfills, stop/go quality gates.
4. **Deployment and service map:** packages, databases, endpoints, secrets, environments, owners.

Suggested tools

Diagramming: draw.io/diagrams.net, Mermaid, Graphviz.
Workflow DAGs: Airflow, Dagster, Prefect.
Data lineage: dbt docs, OpenLineage/Marquez, catalog tools.
Code/package dependencies: pipdeptree, `uv tree`, dependency lockfiles, CI checks.

Choose tools based on project scope; a well-maintained diagram is better than an elaborate unused platform.

How to audit a dependency map

For each node, annotate:

- Owner and environment
- Input / output contract and schema
- Trigger and expected completion time
- Failure detection / retry policy
- Security classification and access
- Downstream consumers and SLA

Then challenge the edges:

- Missing prerequisite or circular dependency?
- Single point of failure?
- What changes if a schema version changes?
- Can this step be repeated safely?
- What is the blast radius of failure?
- Can data be replayed or backfilled?

Practical review

Walk one normal record and one failed/late record through the entire diagram.

Tool-selection matrix: score evidence, not enthusiasm

Criterion	Weight	Option A: simple batch	Option B: streaming stack
Meets freshness goal	30%	3 / 5	5 / 5
Implementation effort	25%	5 / 5	2 / 5
Operational burden	20%	5 / 5	2 / 5
Scalability headroom	15%	3 / 5	5 / 5
Team familiarity	10%	5 / 5	2 / 5
Illustrative weighted score	100%	**4.10 / 5**	**3.15 / 5**

- Scores are **illustrative**, not product facts; derive real scores from prototypes.
- Apply hard constraints (e.g., must meet a 5-second SLA) **before** ranking options.
- Record sensitivity: would the recommendation change if a weight doubled?

Architecture Decision Record (ADR): one decision per page

Example: ADR-003 — batch vs. streaming ingestion

Context: dashboard stakeholders can accept 10-minute freshness.
Options: (A) scheduled API poll + database; (B) Kafka + stream processor.
Decision: prototype A as baseline.
Why: lower operational complexity; requirement currently permits polling.
Risks: rate limits, missed updates, API outages.
Validation: replay test + 95th-percentile end-to-end freshness.
Revisit if: requirements tighten or polling cannot meet reliability goals.

An ADR captures context, alternatives, consequences, evidence, and the conditions for revisiting the choice.

Capstone work plan: artifacts before a full build

Phase	Team action	Evidence / deliverable
1. Define	Describe use case and measurable constraints	One-page requirement / acceptance criteria
2. Map	Sketch logical architecture and four dependency views	Versioned diagrams, assumptions list
3. Compare	Select 2–3 credible architecture/tool options	Decision matrix + preliminary ADR
4. Probe	Implement one end-to-end vertical slice	Timing, cost, quality, recovery test
5. Refine	Review dependencies and failure paths	Updated maps, revised ADR, backlog

Do not start with an installation sprint

First find the riskiest unverified assumption, then design a small experiment that can invalidate it.

In-class capstone exercise (20–25 minutes)

1. **5 min:** Draw sources, ingestion, storage, transformation, and consumers.
2. **5 min:** Add arrows for execution dependencies and mark at least three possible failure points.
3. **5 min:** Identify one simpler and one more sophisticated tool alternative.
4. **5 min:** Draft an ADR with a single falsifiable decision criterion.
5. **5 min:** Exchange maps with another team; have them find missing dependencies.

Share-out prompt

What dependency or hidden assumption did your map reveal, and what evidence would change your tool choice?

What the team should leave with

- One **clear architecture map** with meaningful component and data-flow labels.
- One **dependency DAG** showing ordering, quality gates, and failure handling.
- Two or three **credible tool/architecture alternatives** evaluated against requirements.
- An **ADR** with a decision, trade-offs, and revisit conditions.
- A **small experiment** to validate the highest-risk assumption.

Plan $\rightarrow$ Map $\rightarrow$ Challenge $\rightarrow$ Test $\rightarrow$ Decide